

ToggleMaster — Fase 2: Explicação Detalhada de Tudo que Foi Feito

Este documento registra, passo a passo, tudo que foi executado para reconstruir a infraestrutura, validar os microsserviços, gravar o vídeo e preparar a entrega do Tech Challenge Fase 2. A ideia é que sirva como material de estudo — não só "o que rodamos", mas **por que** cada decisão foi tomada.

Parte 1 — Contexto e ponto de partida

O código dos 5 microsserviços (Go e Python) já estava pronto e testado localmente via Docker Compose. O desafio era **reconstruir toda a infraestrutura AWS do zero**, porque os créditos do Lab anterior do AWS Academy haviam se esgotado — um novo Lab significa uma conta AWS "limpa", sem nada provisionado.

Ordem de trabalho definida: infraestrutura → testes → vídeo → documentação → GitHub. Essa ordem importa porque documentação e GitHub não dependem do Lab estar ativo, então foram deixados por último para não desperdiçar tempo de Lab em tarefas que podem ser feitas com calma depois.

Parte 2 — Autenticação com a AWS

O AWS Academy fornece credenciais **temporárias** (Access Key, Secret Key e Session Token), que expiram em algumas horas e não persistem entre sessões de terminal diferentes.

Tentativa inicial: salvar as credenciais em `~/.aws/credentials`. Isso corrompeu o arquivo repetidamente — o problema era colar o bloco inteiro de uma vez, o que podia introduzir caracteres invisíveis do clipboard.

Solução adotada: usar variáveis de ambiente exportadas diretamente no terminal:

```
bash

export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_SESSION_TOKEN="..."
export AWS_DEFAULT_REGION="us-east-1"
```

Isso precisou ser repetido várias vezes ao longo da sessão, sempre que um terminal novo era aberto ou o token expirava — é uma limitação inerente ao ambiente Academy, não um bug nosso.

Parte 3 — ECR (Elastic Container Registry)

Criamos 5 repositórios, um por microserviço:

```
bash

aws ecr create-repository --repository-name auth-service --region us-east-1
# (repetido para flag-service, targeting-service, evaluation-service, analytics)
```

Depois, autenticamos o Docker no registry, buildamos as 5 imagens localmente e enviamos ((push)) cada uma:

```
bash

aws ecr get-login-password --region us-east-1 | docker login --username AWS --p
docker build -t auth-service:latest ./auth-service
docker tag auth-service:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/auth
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/auth-service:latest
```

Detalhe importante: o (deployments.yaml) do Kubernetes continha o account ID da conta AWS **antiga** hardcoded nas imagens. Isso só foi percebido depois, quando os pods deram (ImagePullBackOff) — corrigido com um (sed) trocando o account ID antigo pelo novo em todas as ocorrências.

Parte 4 — Cluster EKS (via Console AWS)

O AWS Academy **não permite** (eksctl create cluster) nem a criação de novas roles IAM — só é permitido usar a (LabRole) já existente. Por isso, o cluster foi criado manualmente pelo Console AWS.

Decisões na criação do cluster:

Campo	Escolha	Motivo
Cluster IAM Role	(LabRole)	Única role disponível no Academy
Compute config	Custom configuration (não Auto Mode)	Auto Mode tenta criar roles novas automaticamente, o que o Academy bloqueia
Cluster access	Allow cluster administrator access = Sim	Evita precisar configurar (aws-auth) ConfigMap manualmente
Authentication mode	EKS API and ConfigMap	Compatível com os dois métodos de autenticação, mais seguro para o ambiente

Campo	Escolha	Motivo
Endpoint access	Público apenas	Suficiente para o cenário, sem necessidade de rede privada
Control plane egress	AWS managed	Não há topologia de rede customizada que justifique "customer-routed"

O problema do "Auto Mode Compute"

Mesmo escolhendo Custom configuration, uma tela secundária ("Auto Mode Compute") insistia em pedir uma "Node IAM role" com validação:

```
When Compute Config nodeRoleArn is not null or empty, nodePool value(s) must be provided.
```

Causa: essa seção só é exibida quando o modo de compute do cluster ainda está, em algum nível, como Auto Mode — não é possível "limpar" o campo individualmente. A correção real foi voltar e confirmar que o toggle principal de compute estava mesmo em **Custom configuration**; assim que confirmado, a seção inteira de "Auto Mode Compute" desaparece da tela.

Node Group (separado, manual)

Depois do cluster `ACTIVE`, criamos o Node Group manualmente:

- **Node IAM role:** `LabRole`
- **AMI:** Amazon Linux 2023
- **Instância:** `t3.medium` (2 vCPU / 4GB RAM — suficiente para rodar 5 serviços + Nginx Ingress + Metrics Server sem os pods ficarem `Pending` por falta de recursos)
- **Auto Scaling:** mínimo 1, desejado 2, máximo 4

Conectamos o `kubectl` local ao cluster:

```
bash
aws eks update-kubeconfig --region us-east-1 --name togglemaster-cluster
```

Parte 5 — RDS, ElastiCache, DynamoDB e SQS (em paralelo ao EKS)

Como a criação do cluster EKS leva 10-15 minutos, aproveitamos esse tempo para provisionar os outros recursos em paralelo, sem depender do cluster estar pronto.

RDS (3 instâncias PostgreSQL)

Cada banco (auth-db, flag-db, targeting-db) precisou de:

1. Um Security Group liberando a porta `5432` para o range da VPC (`172.31.0.0/16`)
2. A instância em si:

```
bash

aws rds create-db-instance \
  --db-instance-identifier auth-db \
  --db-instance-class db.t3.micro \
  --engine postgres \
  --master-username postgres \
  --master-user-password 'ToggleMaster123!' \
  --allocated-storage 20 \
  --vpc-security-group-ids sg-xxxxxxx \
  --publicly-accessible \
  --backup-retention-period 0
```

Nota: o `!` na senha precisou de aspas simples no bash — com aspas duplas, o shell tenta expandir `!` como referência de histórico de comandos (`event not found`).

ElastiCache (Redis)

Mesmo padrão: Security Group liberando porta `6379`, depois:

```
bash

aws elasticache create-cache-cluster \
  --cache-cluster-id togglemaster-redis \
  --engine redis \
  --cache-node-type cache.t3.micro \
  --num-cache-nodes 1 \
  --security-group-ids sg-xxxxxxx
```

DynamoDB

Antes de criar a tabela, verificamos no código do `analytics-service` (`app.py`) qual campo era usado como chave primária no `put_item` — o código usa `event_id` (String), gerado via `uuid.uuid4()`. Criamos a tabela com esse schema exato:

```
bash
```

```
aws dynamodb create-table \  
  --table-name ToggleMasterAnalytics \  
  --attribute-definitions AttributeName=event_id,AttributeType=S \  
  --key-schema AttributeName=event_id,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST
```

SQS

```
bash  
  
aws sqs create-queue --queue-name togglemaster-queue
```

Parte 6 — Kubernetes: Secrets e ConfigMaps

Antes de aplicar qualquer manifest, geramos os `Secrets` do Kubernetes com os endpoints reais dos recursos AWS recém-criados (RDS, Redis, SQS, DynamoDB), todos codificados em base64 (exigência do formato `Secret` do Kubernetes).

```
bash  
  
echo -n "postgresql://postgres:SENHA@auth-db.xxxx.rds.amazonaws.com:5432/postgr
```

Aplicamos:

```
bash  
  
kubectl create namespace togglemaster  
kubectl apply -f k8s/secrets.yaml  
kubectl apply -f k8s/configmap.yaml
```

Parte 7 — Metrics Server (e o conflito com o add-on gerenciado)

Instalamos inicialmente via manifest da comunidade:

```
bash  
  
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/lat
```

Isso gerou um erro de conflito (`Deployment "metrics-server" is invalid`) — já existia um Deployment antigo do cluster anterior. Resolvido deletando o Deployment antigo e reaplicando.

Diagnóstico, passo a passo:

1. `kubectl get apiservice v1beta1.metrics.k8s.io -o yaml` → revelou `MissingEndpoints`
2. `kubectl get svc metrics-server -n kube-system -o yaml` → mostrou que o `selector` exigia 3 labels específicas
3. `kubectl get pods -n kube-system -l k8s-app=metrics-server --show-labels` → o pod só tinha 1 das 3 labels exigidas

Solução: deletar o Deployment manual e deixar o add-on gerenciado da AWS assumir sozinho, forçando uma resincronização:

```
bash

kubectl delete deployment metrics-server -n kube-system
aws eks update-addon --cluster-name togglemaster-cluster --addon-name metrics-server
```

Depois disso, `kubectl top pods` passou a funcionar normalmente.

Parte 8 — Nginx Ingress Controller

Instalado via Helm:

```
bash

helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install nginx-ingress ingress-nginx/ingress-nginx --namespace togglemaster
```

Isso provisiona automaticamente um **Load Balancer** na AWS. O endereço público (`EXTERNAL-IP`) foi confirmado com:

```
bash

kubectl get svc -n togglemaster nginx-ingress-nginx-controller
```

Parte 9 — Deployments, Services e o bug do `ImagePullBackOff`

Aplicamos os manifests dos 5 microserviços:

```
bash

kubectl apply -f k8s/deployments.yaml
kubectl apply -f k8s/services.yaml
```

Todos os pods entraram em `ImagePullBackOff`/`ErrImagePull`. Causa: o `deployments.yaml` referenciava o account ID da conta AWS **antiga** (diferente do Lab atual). Corrigido com:

```
bash

sed -i 's/<account-id-antigo>/<account-id-novo>/g' k8s/deployments.yaml
kubectl apply -f k8s/deployments.yaml
```

Depois da correção, o Kubernetes fez um **rolling update** automaticamente: criou pods novos com a imagem certa e só desligou os antigos quando os novos passaram no *readiness probe* — por isso, por um instante, havia o dobro de pods listados (metade `Running`, metade `ImagePullBackOff`) até a transição terminar sozinha.

Parte 10 — Ingress e o primeiro teste ponta a ponta

Aplicamos o `ingress.yaml`, que define o roteamento por prefixo (`/auth`, `/flags`, `/rules`, `/evaluate`, `/analytics`) usando a anotação `nginx.ingress.kubernetes.io/rewrite-target: /$1` — que **remove** o prefixo antes de encaminhar a requisição ao serviço interno.

Testamos com `curl` no endereço do Load Balancer:

```
bash

curl http://<load-balancer>/auth/health
```

Retornou `200 OK` — confirmando que a cadeia completa (Load Balancer → Nginx Ingress → Service → Pod) estava funcionando.

Parte 11 — HPA (Horizontal Pod Autoscaler)

O que é

O HPA cria (ou remove) réplicas de um Deployment automaticamente, com base em uma

métrica — no nosso caso, uso médio de CPU. É escalonamento **horizontal** (mais cópias do mesmo pod), diferente do escalonamento **vertical** (dar mais recursos ao mesmo pod).

```
bash

kubectl apply -f k8s/hpa.yaml
kubectl get hpa -n togglemaster
```

Inicialmente, o `TARGETS` ficou em `<unknown>/70%` — causa raiz: o problema do Metrics Server duplicado (ver Parte 7). Depois de corrigido, o HPA passou a mostrar métricas reais de CPU.

Parte 12 — Debug de autenticação entre serviços (401 e o bug do `placeholder`)

Ao testar o endpoint `/evaluate`, recebíamos:

```
json

{"error": "Erro interno ao avaliar a flag"}
```

Investigação, passo a passo:

1. **Rota correta:** descobrimos, testando com `curl -v`, que a rota real era `/evaluate/evaluate` (não `/evaluate/`) — porque o rewrite do Ingress remove o prefixo `/evaluate`, mas a rota interna do Go também se chama `/evaluate`, então era preciso "duplicar" no caminho externo.
2. **Erro real:** os logs do `evaluation-service` mostraram `flag-service retornou status 401`.
3. **Causa:** o `evaluation-service` chama o `flag-service`, que por sua vez valida a chave de API contra o `auth-service` (endpoint `/validate`). A variável `SERVICE_API_KEY` estava vazia — não existia ainda uma chave de API válida gerada.
4. **Criação da chave:** o `auth-service` tem uma rota `POST /admin/keys`, protegida por uma `MASTER_KEY`. Geramos e usamos essa chave para criar uma API key nova:

```
bash

curl -X POST http://<load-balancer>/auth/admin/keys \
  -H "Authorization: Bearer <MASTER_KEY>" \
  -d '{"name": "service-to-service"}'
```

5. **Erro seguinte:** `"Erro ao salvar a chave"` → os logs mostraram `relation "api_keys" does not exist` — o banco RDS estava **vazio** (recriado do zero, sem schema). Resolvido aplicando os scripts `init.sql` de cada serviço via `psql` (depois

de liberar o IP local no Security Group do RDS, já que por padrão só a VPC tinha acesso).

6. **Ainda 401**, mesmo com a chave criada: comparando o hash SHA-256 da chave esperada com o hash recebido nos logs do `auth-service`, os valores **não batiam**. Investigando o `deployments.yaml`, encontramos a causa:

```
yaml
env:
  - name: SERVICE_API_KEY
    value: "placeholder"      # <- isso sobrescrevia o valor do Secret!
  - name: AWS_SQS_URL
    valueFrom:
      secretKeyRef: ...
```

No Kubernetes, variáveis definidas explicitamente em `env:` têm prioridade sobre `envFrom: secretRef:`, mesmo vindo depois na lista. Removemos as 2 linhas do `env:` explícito, deixando o `envFrom` (que já injetava todas as chaves do Secret, incluindo a correta) prevalecer.

Depois desse ajuste, o teste completo funcionou:

```
{"flag_name":"teste","user_id":"teste","result":false}.
```

Parte 13 — IMDS Hop Limit (credenciais da LabRole dentro dos pods)

Ao testar o pipeline assíncrono (SQS → DynamoDB), os logs mostravam:

```
NoCredentialProviders: no valid providers in chain
Unable to locate credentials
```

Explicação técnica

Pods dentro do Kubernetes herdam credenciais AWS da role do nó EC2 onde estão rodando, via **IMDS** (Instance Metadata Service — endpoint interno `169.254.169.254`). Por padrão, o **hop limit** do IMDS é `1`, o que só permite acesso de processos rodando **diretamente** na instância. Um pod está "um salto de rede" além disso (dentro de uma rede virtual de container), então a requisição não completa.

Solução

Aumentar o hop limit para `2` em cada instância dos nós:

```
bash
```

```
aws ec2 modify-instance-metadata-options \
  --instance-id i-xxxxxxx \
  --http-put-response-hop-limit 2 \
  --http-endpoint enabled
```

Depois disso, reiniciamos os Deployments afetados:

```
bash
kubectl rollout restart deployment evaluation-service analytics-service -n togg
```

E confirmamos o item aparecendo no DynamoDB via `aws dynamodb scan`.

Parte 14 — Calibração do HPA do `analytics-service`

No roteiro do vídeo, planejamos demonstrar o HPA do `analytics-service` escalando sob carga. Testamos enviando mensagens manualmente à fila SQS:

- 50 mensagens → pico de CPU: 9%/70% (muito baixo)
- 500 mensagens → pico de CPU: 67%/70% (quase lá)
- 2.000 mensagens → pico de CPU: ~60%/70% (platô, não passava disso)

Por que a CPU não passava de ~60%?

O worker do `analytics-service` processa mensagens **sequencialmente** (`sqs_worker_loop`), e cada mensagem envolve uma chamada de rede ao DynamoDB. Isso torna o workload **I/O-bound** (gasta tempo esperando rede), não **CPU-bound** (gasta tempo computando) — por isso a CPU satura num patamar bem abaixo de 100%, independente de quantas mensagens existem na fila.

Solução

Como o valor de 70% sugerido no desafio é só um **exemplo**, recalibramos o alvo para 50%, com base no comportamento real observado:

```
bash
kubectl patch hpa analytics-service-hpa -n togglemaster --type='json' \
  -p='[{"op":"replace","path":"/spec/metrics/0/resource/target/averageUtilizati
```

Depois desse ajuste, o HPA passou a escalar corretamente sob a mesma carga de teste.

Parte 15 — Roteiro e gravação do vídeo

Preparamos um roteiro dividido em 8 blocos, com tempo estimado por bloco, comandos exatos prontos para copiar/colar, e falas sugeridas para cada parte:

1. Abertura
2. Docker Compose local
3. Cluster Kubernetes na nuvem
4. Nginx Ingress funcionando
5. Escalabilidade: gerando carga no `evaluation-service` (Apache Bench)
6. Fila SQS + escalabilidade do `analytics-service`
7. Explicação da arquitetura e desafios técnicos
8. Encerramento

A gravação foi feita **por blocos separados** (mais seguro do que tentar gravar tudo de uma vez), depois unidos com `ffmpeg`:

```
bash
```

```
ffmpeg -f concat -safe 0 -i lista.txt -c copy video_final.mp4
```

Resultado: vídeo final de **18min52s**, dentro do limite de 20 minutos.

Parte 16 — GitHub

Problemas encontrados na primeira tentativa

1. **Repositórios Git aninhados:** cada pasta de serviço tinha seu próprio `.git` interno (resquício de como o código foi originalmente baixado), fazendo o Git tratá-las como submódulos vazios. Resolvido removendo os `.git` internos antes do commit.
2. **Vazamento de credenciais no histórico:** um commit anterior (de testes) continha o `secrets.yaml` real, com credenciais da infraestrutura **antiga** (já destruída, mas mesmo assim uma prática ruim manter no histórico). Resolvido recomeçando o repositório do zero (`rm -rf .git && git init`), com o `.gitignore` já valendo desde o primeiro commit.
3. **Autenticação:** o GitHub não aceita mais senha normal via HTTPS desde 2021 — foi necessário gerar um **Personal Access Token (PAT)** e usá-lo no lugar da senha.
4. **Push rejeitado (`fetch first`):** depois de editar o README direto pela interface web do GitHub, o repositório local ficou desatualizado em relação ao remoto. Resolvido com `git pull origin main --rebase` antes do próximo `push`.

`.gitignore` criado para proteger segredos

```
k8s/secrets.yaml
*.env
.env*
aws-creds.sh
*credentials*
```

Parte 17 — Documentação final

- **README.md:** badges, sumário navegável, tabela de participantes, arquitetura, diagrama embutido, infraestrutura AWS, HPA, desafios técnicos.
- **Diagrama de arquitetura:** gerado com Graphviz (não matplotlib — o layout automático do Graphviz produziu um resultado muito mais organizado, com as camadas — Entrada, Microserviços, Armazenamento — bem delimitadas, e as chamadas internas do `evaluation-service` isoladas numa nota lateral em vez de cruzar por cima do diagrama estrutural).
- **Relatório de entrega em PDF:** participantes, links, resumo, arquitetura com diagrama, ambiente local, infraestrutura AWS, escalabilidade, evidências reais de teste (os números coletados durante a calibração do HPA), desafios técnicos, checklist de entregáveis e conclusão.
- **Versão .docx** do mesmo relatório, gerada para edição fácil via Word Online.

Resumo dos principais aprendizados técnicos

1. **AWS Academy tem restrições reais de IAM** que forçam decisões de arquitetura diferentes do "caminho feliz" documentado oficialmente (sem `eksctl`, sem IRSA/KEDA, sem criação de roles novas).
2. **Add-ons gerenciados do EKS podem conflitar com instalações manuais** — sempre vale checar `aws eks list-addons` antes de aplicar manifests da comunidade para componentes centrais como Metrics Server.
3. **IMDS hop limit é uma pegadinha comum em EKS** sempre que pods precisam de credenciais AWS sem usar IRSA.
4. **Prioridade de variáveis de ambiente no Kubernetes:** `env:` explícito sempre vence `envFrom:`, mesmo definido depois na lista — um detalhe fácil de esconder um bug.
5. **HPA por CPU nem sempre reflete a carga real do sistema** — workloads I/O-bound podem nunca atingir alvos de CPU altos, exigindo calibração baseada em teste empírico, não em valores padrão de documentação.

6. **Segredos no histórico do Git são permanentes** até que o histórico seja reescrito — remover o arquivo num commit novo não é suficiente.